If we assume that the average length of the strings held by the tree is `L`, the expected value for `S(n)`, the space needed to store the tree, is proportional to `n*L`; if the maximum length for strings is denoted with `m`, then `S(n) = O(n*m)` is a strict upper bound for the worst case.

Likewise, if we look at the running time of the search method, we see that we can't ignore the strings' length anymore: for instance, a call to `search("antic")` on the tree in figure 6.1 would start from the root, compare "`antic`" to "`anthem`", which would require at least 4 characters to be compared before verifying the two words are not the same, then we would move to the right branch and again compare the two strings, "`antic`" and "`antique`" (5 more character-to-character comparisons), and since they don't match traverse the left subtree now, and so on.

Therefore, a call to search would require, worst case, `T(n) = O(log(n)*m)` comparisons.

6.1.2 Compression is the Key

This quick analysis shows Using a tree is not ideal, neither space-wise nor performance-wise; if we look more closely at the tree in figure 6.1, we can see that there sure seems to be a lot of overhead: all words start with the character '`a`', but still this is stored once for every node of the subtree in the illustration, and for each step in the traversal of the tree, it will be compared with the text that is being searched (or inserted).